Data Structures and Algorithm

COURSE CODE: CS-201

TOPIC: MULTI DIMENSIONAL ARRAY

Goals for today

To study:

- Multi-dimensional Arrays

CLO Covered

CLO2: **Apply** and design various data structures methods for specified applications.

Multidimensional Array

One-Dimensional Array

Two-Dimensional Array

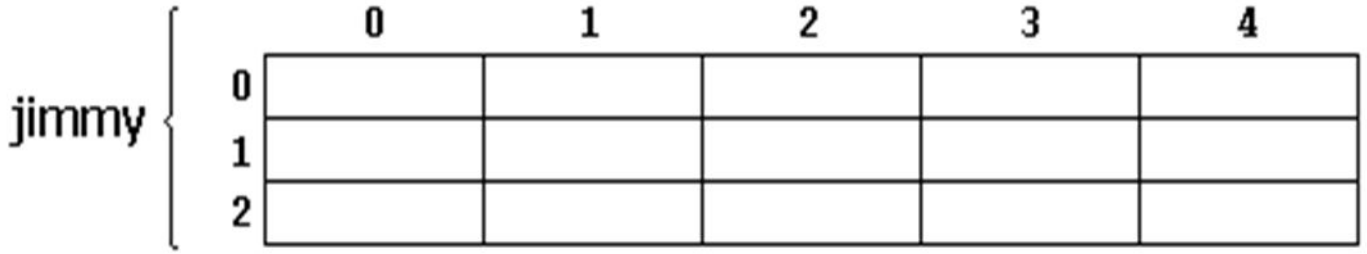
Three-Dimensional Array

Some programming Language allows as many as 7 dimension

Multidimensional Array

- Multidimensional arrays can be described as "arrays of arrays". For example, a bi-dimensional array can be imagined as a bi-dimensional table made of elements, all of them of a same uniform data type.
- jimmy represents a bi-dimensional array of 3 per 5 elements of type int. The way to declare this array in C++ would be:

```
int jimmy [3][5];
```



Multi-Dimensional Arrays

- **A multi-dimensional array is like several identical arrays put together. It is useful for storing multiple sets of data.**

- Two-dimensional arrays, which are sometimes called *2D arrays*, can hold multiple sets of data. It is best to think of a two dimensional array as having rows and columns of elements.

Two-Dimensional Arrays Declaration

- C++ allows arrays with multiple index values
  - **int page [30] [100];**
    declares an array of integers named page
    - page has two index values:
      The first ranges from 0 to 29
      The second ranges from 0 to 99
    - Each index in enclosed in its own brackets
    - Page can be visualized as an array of
      30 rows and 100 columns

Index Values of page

- The indexed variables for array page are
  page[0][0], page[0][1], …, page[0][99]
  page[1][0], page[1][1], …, page[1][99]

  …
- page[29][0], page[29][1], … , page[29][99]

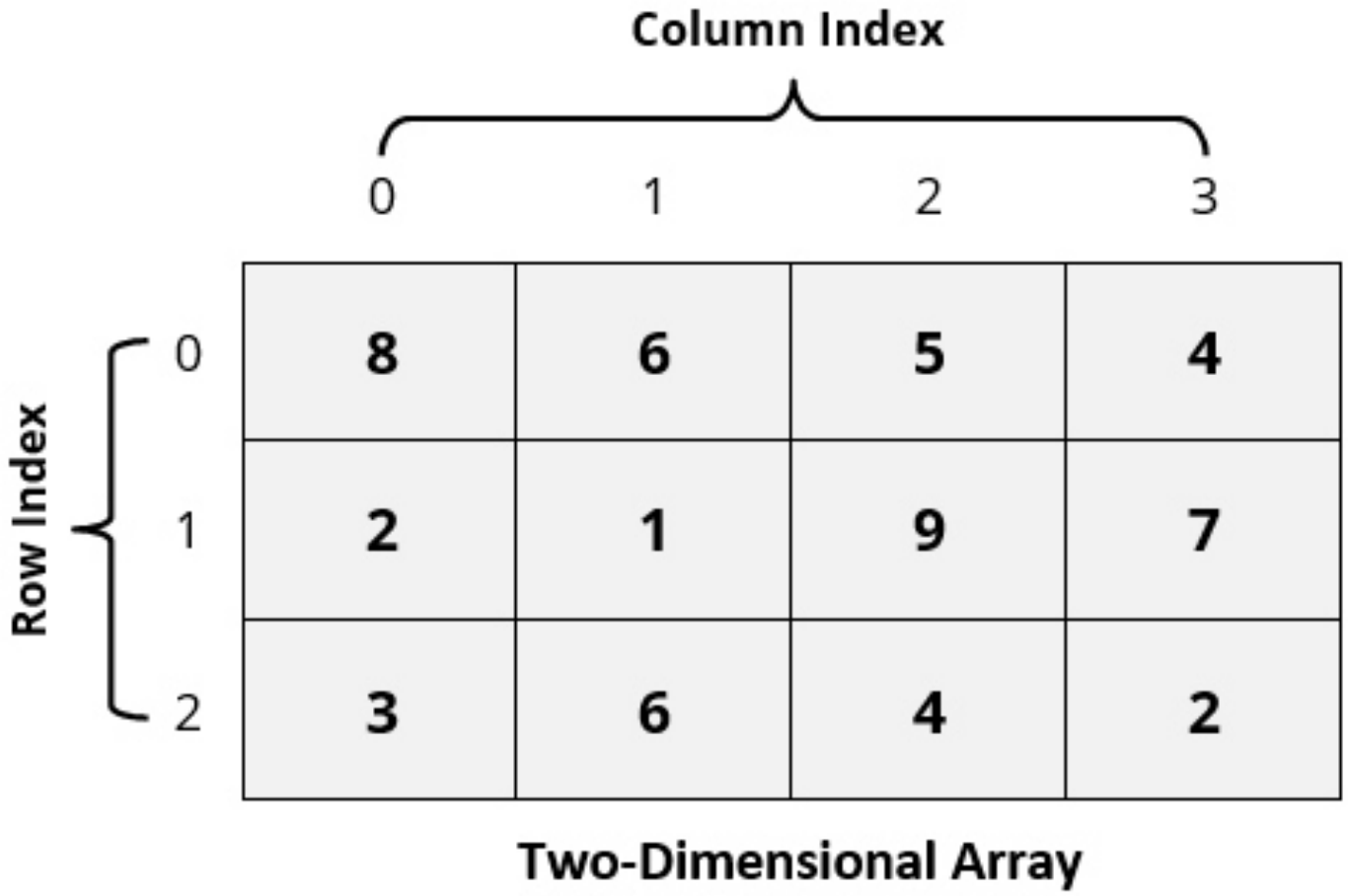
Two-Dimensional Arrays Initialization

```
int hours[3][2] = {  {8, 5}, {7, 9}, {6, 3}  } ;
```

The same definition could also be written as:

```
int hours[3][2] = {  {8, 5},
                     {7, 9},
                     {6, 3}  } ;
```

Two-Dimensional Array



Two-Dimensional Array

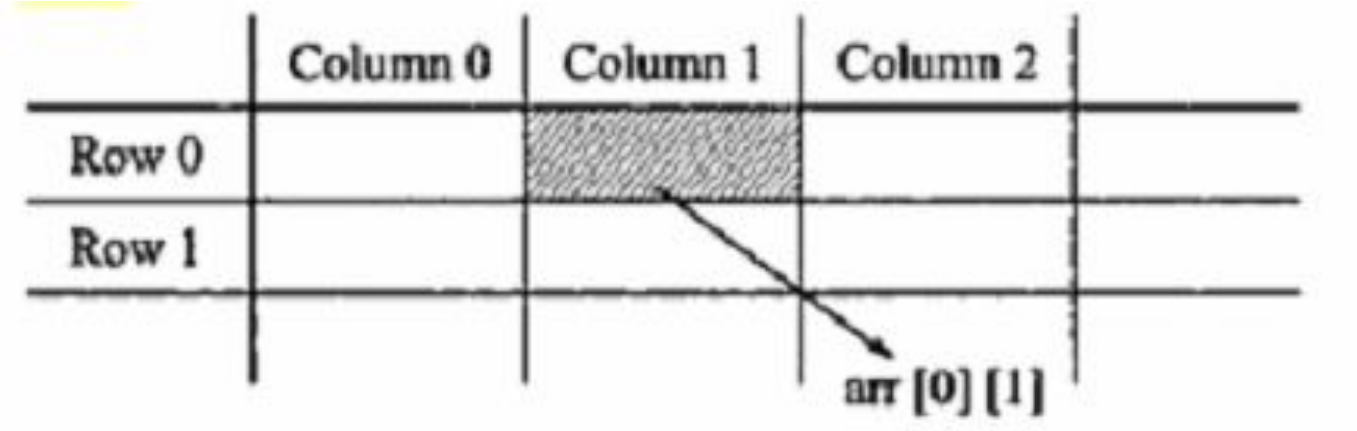
A Two-Dimensional **m x n** array **A** is a collection of **m . n** data elements such that each element is specified by a pair of integer (such as J, K) called subscript with property that

$1 \leq J \leq m$ and **$1 \leq K \leq n$**

The element of **A** with first subscript **J** and second subscript **K** will be denoted by **$A_{J,K}$** or **A[J][K]**

2D Arrays

The elements of a 2-dimensional array a is shown as below



2D Array

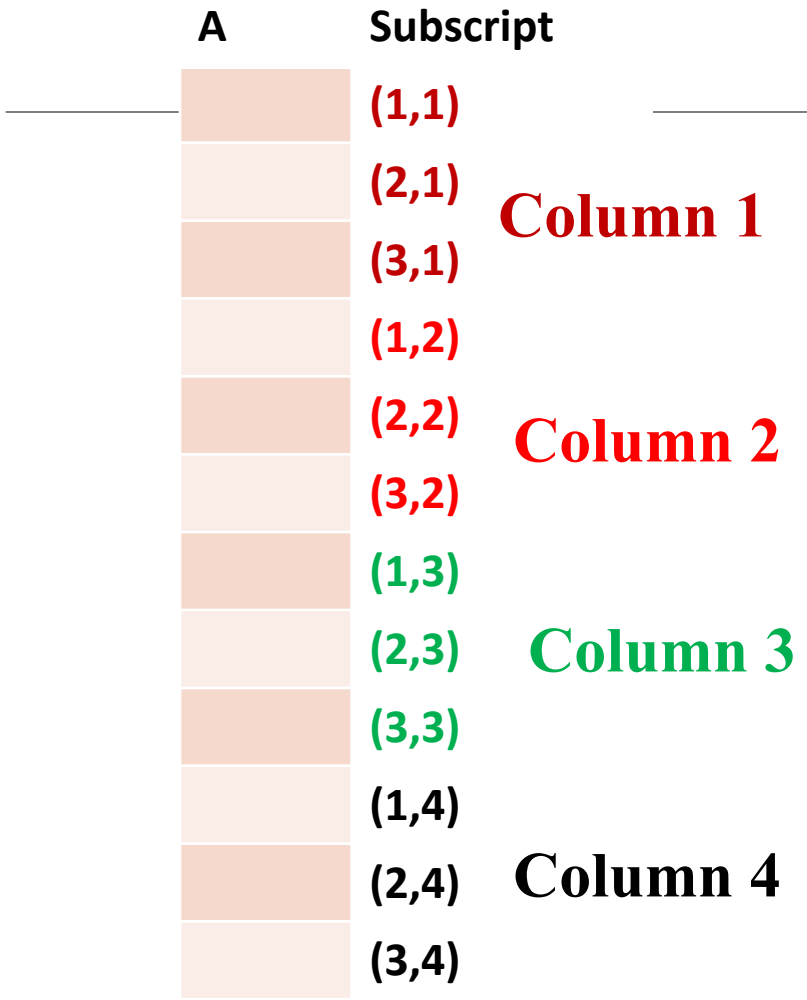
Let **A** be a two-dimensional array **m x n**

The array **A** will be represented in the memory by a block of **m x n** sequential memory location

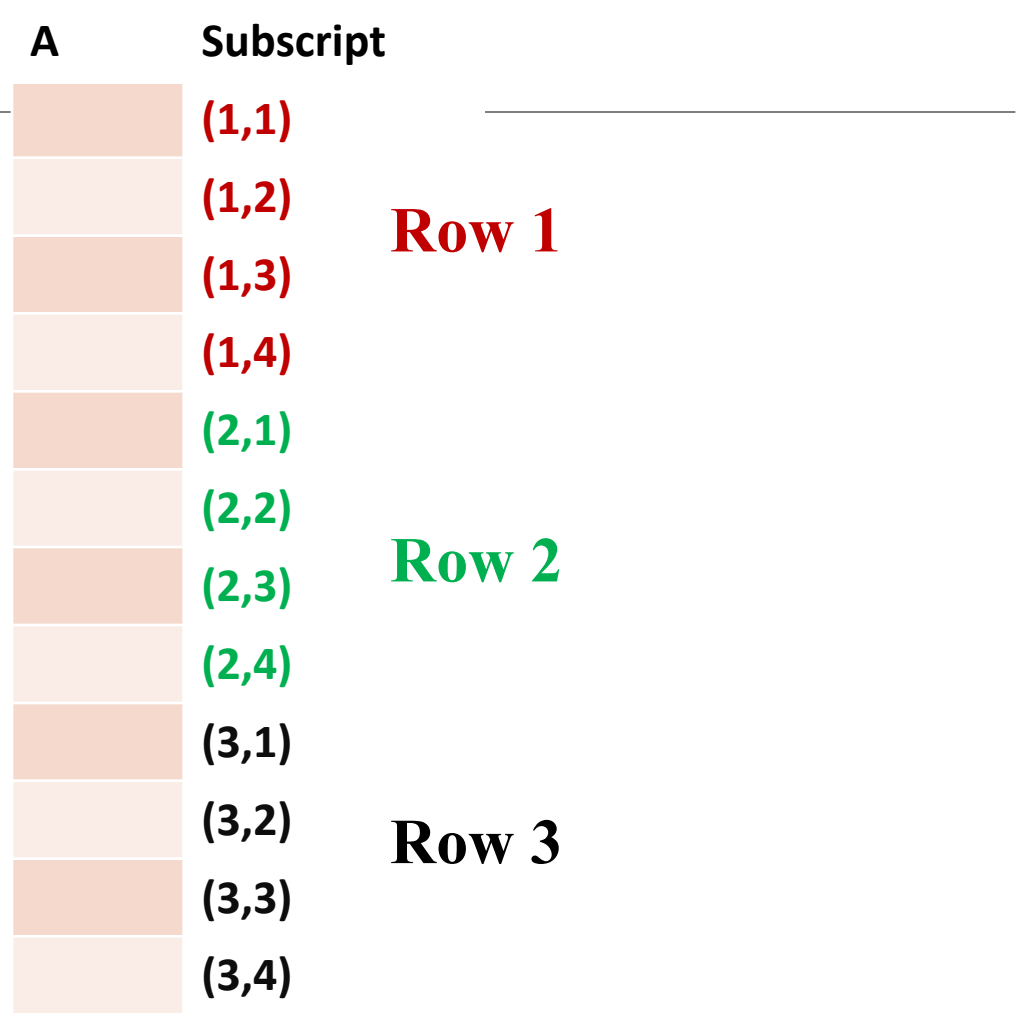
Programming language will store array **A** either

- **Column by Column** (Called Column-Major Order)
- **Row by Row** (Called Row-Major Order)

Representation of Two-Dimensional Arrays in Memory



Column-Major Order



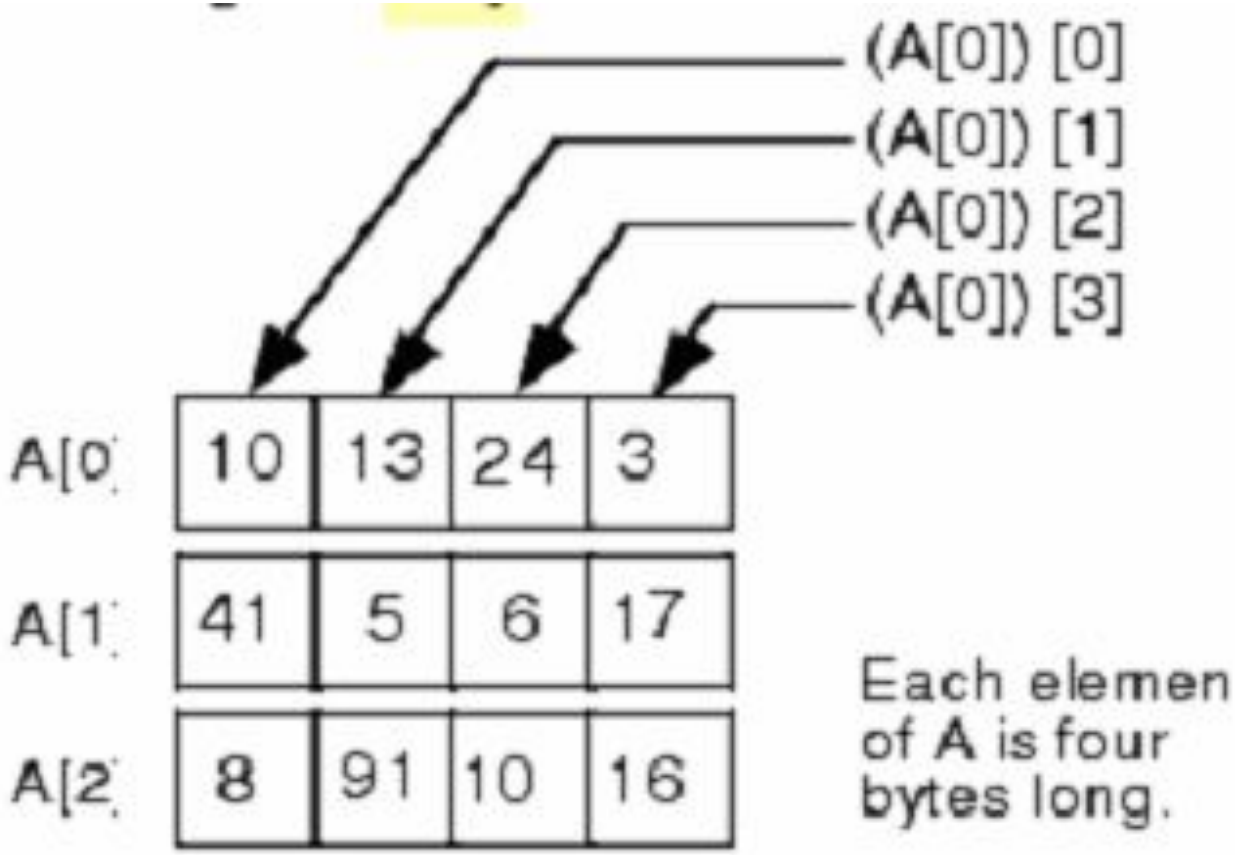
Row-Major Order

Representation of 2D Arrays in Memory

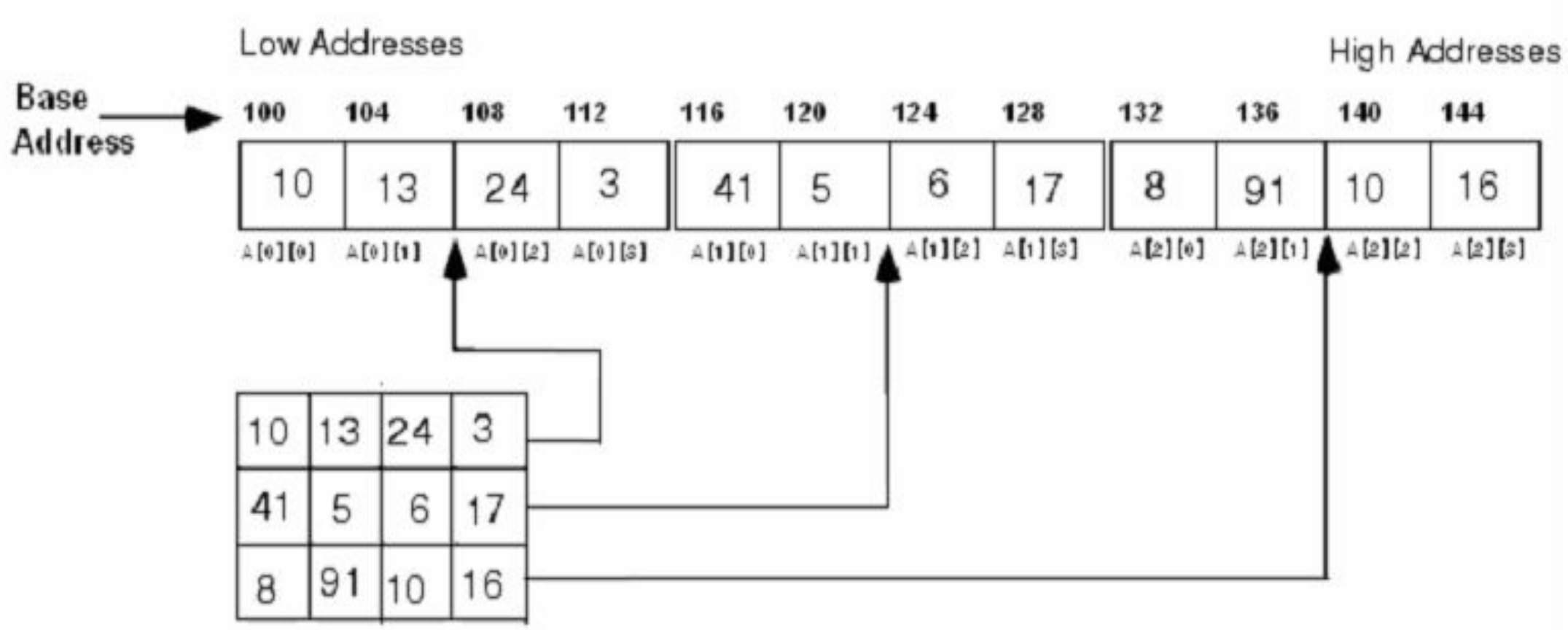
For Example:

```
int  A[3][4] = {10,13,24,3,
                41,5,6,17,
                8,91,10,16};
```

then how this 2-D array will be presented externally and in the memory of the computer using Row-Major-Order



Representation of 2D Arrays in Memory



Address of Elements in Multi Dimensional Array

- For Linear Array LA, the computer does not keep track of address LOC(LA[K]) of every element LA[K] of LA, but does **keep track of Base(LA),** the address of first element of LA.
- For 1D Array;

$$LOC(LA[K]) = Base(LA) + w(K - LB)$$

To find the address of LA[K] in time independent of K. w is number of words per memory block.

Address of Elements in Multi Dimensional Array

For 2D m x n array A, computer keeps track of Base(A), the address of first element A[1,1] of A, computes the address LOC(A[J, K]) of A[J, K] using;

LOC(A[J,K]) of A[J,K]

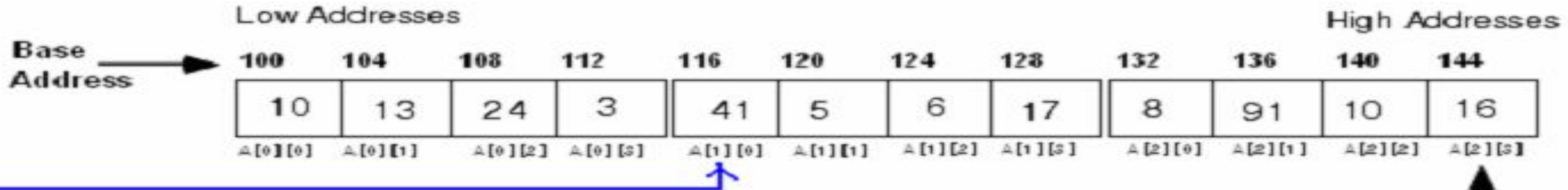
Column-Major Order

LOC(A[J,K]) = Base(A) + w[M(K-LB) + (J-LB)]

Row-Major Order

LOC(A[J,K]) = Base(A) + w[N(J-LB) + (K-LB)]

Row Major Order



Formula to calculate/find the address of A[j][k]th element of a 2-D array of M × N dimension is

$$A[j][k] = base(A) + W[N(j - Row\_LowBound) + (k - Col\_LowBound)]$$

where W = size of element
N = number of columns

Let us assume that the base address of the array matrix is 100. Since W= 4 (as array is of integer type whose size is 4), therefore, according to the formula, address of (2, 3)th element in the array matrix will be

$$\begin{aligned} LOC(A[2][3]) &= 100 + 4[4(2-0)+(3-0)] \\ &= 100 + 4[8 + 3] \\ &= 100 + 4[11] \\ &= 100 + 44 \\ &= 144 \end{aligned}$$

$$\begin{aligned} LOC(A[1][0]) &= 100 + 4[4(1-0)+(0-0)] \\ &= 100 + 4[4 + 0] \\ &= 100 + 4[4] \\ &= 100 + 16 \\ &= 116 \end{aligned}$$

Stored as Column-Major-Order

10	13	24	3
41	5	6	17
8	91	10	16

Similarly, for the column major order representation, let us consider the same matrix.

int A[3] [4] = {10,13,24,3, 41,5,6,17,8,91,10,16};

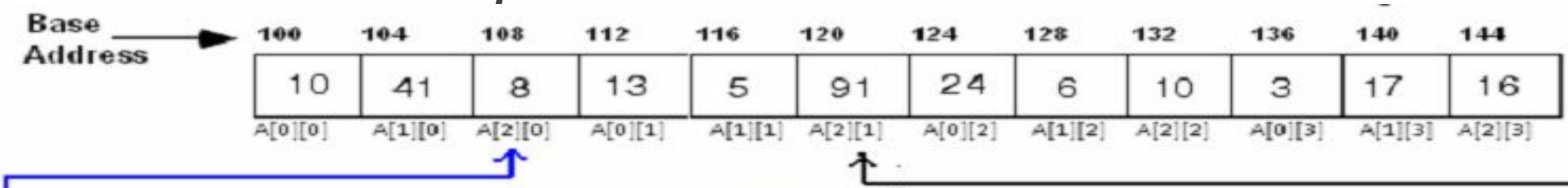
Here M=3 and N = 4

$$A(j, k) = Base(A) + W[M(k-1) + (j-1)]$$

Low Addresses — High Addresses

Base Address →

Column Major Order



Formula to calculate/find the address of A[j][k]th element of a 2-D array of M × N dimension is

$$A[j][k] = base(A) + W[M(k - Col\_LowBound) + (j - Row\_LowBound)]$$

where W = size of element
M = number of Rows

Let us assume that the base address of the array matrix is 100. Since W= 4 (as array is of integer type whose size is 4). therefore, according to the formula, address of (2, 3)th element in the array matrix will be

$$\begin{aligned} LOC(A[2][1]) &= 100 + 4[3(1-0)+(2-0)] \\ &= 100 + 4[3 + 2] \\ &= 100 + 4[5] \\ &= 100 + 20 \\ &= 120 \end{aligned}$$

$$\begin{aligned} LOC(A[2][0]) &= 100 + 4[3(0-0)+(2-0)] \\ &= 100 + 4[0 + 2] \\ &= 100 + 4[2] \\ &= 100 + 8 \\ &= 108 \end{aligned}$$

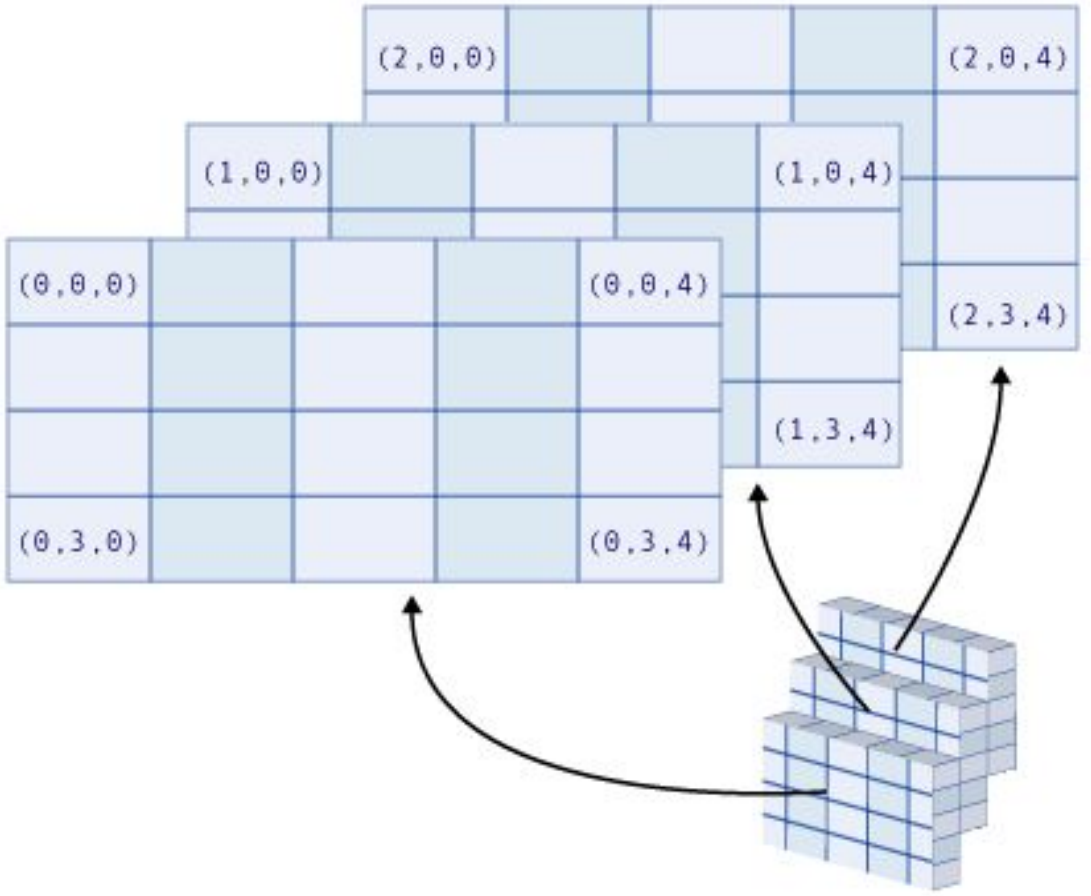
Arrays with Three or More Dimensions

C++ does not limit the number of dimensions that an array may have. It is possible to create arrays with multiple dimensions, to model data that occur in multiple sets.

Here is an example of a three-dimensional array definition:

```
double seats[3][4][5];
```

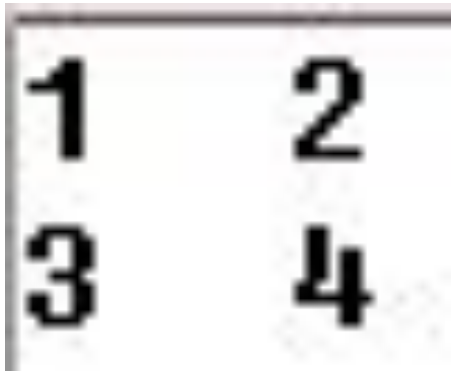
This array can be thought of as three sets of four rows, with each row containing five elements.



```
#include <iostream.h>
int main()
{
int a[5][2] = { {0,1}, {2,3}, {4,5}, {6,7}, {8,9}};
for (int i = 0; i<5; i++)
for (int j=0; j<2; j++)
{
cout << "a[" << i << "][" << j << "]: ";
cout << a[i][j]<< endl;
}
return 0;
}
```

```
a[0][0]: 0
a[0][1]: 1
a[1][0]: 2
a[1][1]: 3
a[2][0]: 4
a[2][1]: 5
a[3][0]: 6
a[3][1]: 7
a[4][0]: 8
a[4][1]: 9
```

```
#include <iostream.h>
void main()
{
int t,i, nums[2][2];
for(t=0; t < 2; ++t)
{
for(i=0; i < 2; ++i)
{
nums[t][i] = (t*2)+i+1;
cout << nums[t][i]<<"  ";
}
cout << “\n”;
}
}
```



Algorithm for addition of two matrices

No. of rows and cols of A=No. of rows and cols of B

```
Repeat for (i=0; i<rA; i++)
 {
Repeat for (j=0; j<cB; j++)
 {
C[i][j]=A[i][j]+B[i][j];
 }
 }
```

Algorithm for multiplication of two matrices

No. of cols of A=No. of rows of B

```
Repeat for (i=0; i<rA; i++)
 {
Repeat for (j=0; j<cB; j++)
 {
Repeat for (k=0; k<cA; k++)
 {
C[i][j]= C[i][j]+A[i][k]*B[k][j];
 }
 }
 }
```

Questions?